

SURVEY

AI-Driven Software Test Automation: An AI4SE-Oriented Survey of Techniques, Tools, and Challenges

AAZAADE FARAJI¹, (Student Member, IEEE), AND NUNO POMBO², (Senior Member, IEEE)

¹Universidade da Beira Interior, 6201-001 Covilhã, Portugal

²Instituto de Telecomunicações, Universidade da Beira Interior, 6201-001 Covilhã, Portugal

Corresponding author: Nuno Pombo (ngpombo@ubi.pt)

This work was supported in part by the Foundation for Science and Technology (FCT), Portugal, through the project “Intelligent System for Mapping Cultural Heritage Markers in Play Behaviour,” under Grant PRT/BD/155023/2023; and in part by FCT under Project UIDB/50008/2020.

ABSTRACT The growing complexity of software systems and the need for more rapid, high-quality software releases have created the need for intelligent and automated testing mechanisms. Drawing on AI-driven test automation of software in the larger context of Artificial Intelligence for Software Engineering, this survey discusses how artificial intelligence methods are used to augment and automate multiple stages of the software test cycle, ranging from test planning, generation, execution, repair, and maintenance. We offer a systematic overview of 76 industrial tools and peer-reviewed research and examine the contribution of Machine Learning, Deep Learning, Large Language Models, Reinforcement Learning, and other paradigms of artificial intelligence to modern test automation pipelines. The paper suggests a lifecycle-oriented taxonomy, aligning industry with research with industrial practices, and offers structured visual analyses to map current trends, limitations, and emerging research opportunities. Special focus is awarded to untapped areas such as Explainable Artificial Intelligence, Natural Language Processing-based test scripting, and hybrid AI strategies. This survey also acts as a ready reference guide for researchers and practitioners towards AI-driven software testing under the framework of Artificial Intelligence for Software Engineering.

INDEX TERMS Artificial intelligence, automated software testing, deep learning, intelligent system, large language models, machine learning, reinforcement learning, software testing, test automation, test case generation.

I. INTRODUCTION

Software testing is a crucial and mandatory phase of the Software Development Lifecycle (SDLC) to guarantee software quality, reliability, and performance [1], [2]. With the increase in complexity in contemporary software systems, including dynamic user interfaces, distributed architectures, real-time computing, and Artificial Intelligence (AI) components, traditional testing techniques are faced with huge scalability and flexibility challenges [3], [4]. Conventional rule-based frameworks (e.g., Selenium, JUnit) rely on fragile,

manually written scripts that are costly to maintain and insufficiently agile as systems evolve [5], [6], [7], [8], [9], [10]. To overcome these challenges, the branch of *Artificial Intelligence for Software Engineering* (AI4SE) has grown, leveraging AI methods to automate and streamline software engineering tasks across the entire SDLC at scale [11], [12]. AI4SE encompasses a wide range of activities such as requirements analysis, code generation, refactoring, defect prediction, and most importantly, software testing [11].

Falling under this broad umbrella, *AI-based software test automation* has received significant attention as an emerging technique for breaching the confines of conventional testing methods [12]. **AI-based software test automation** is the

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana¹.

application of AI technology, such as Machine Learning (ML) [13], Deep Learning (DL) [14], Large Language Models (LLMs) [15], and Reinforcement Learning (RL) [16], to automate and optimize diverse software testing activities. They encompass test case generation, adaptive execution, failure identification, test repair, optimization, and continuous integration in Development and Operations (DevOps) pipelines [11], [12], [17]. As opposed to static rule-based automation, AI-based approaches can learn from code, user input, run history, and behavior patterns, allowing for intelligent, context-aware test flows that adapt according to system complexity [11], [17], [18].

Recent breakthroughs in AI technologies have motivated researchers to develop new testing paradigms. LLMs (e.g., Generative Pre-trained Transformer (GPT)-4, Large Language Model Meta AI (LLaMA)) have been shown to generate human-readable test cases from natural language specs with encouraging results [19], [20]. RL agents are becoming more prevalent to autonomously navigate graphical user interfaces (GUIs), identify edge cases, and optimize test execution orders [21], [22], [23].

Furthermore, DL models aid fault prediction, log analysis, and semantic test repair [17], [24], [25]. Such advances reveal the revolutionary potential of AI in software testing but also introduce new challenges around scalability, explainability, data dependency, and integration with industrial Continuous Integration/Continuous Deployment (CI/CD) pipelines [9], [11], [26], [27].

A. MOTIVATION AND SCOPE

In spite of the presence of some survey papers on AI usage for software testing [27], [28], the majority of these papers are dedicated to individual AI techniques (e.g., defect prediction, visual validation) or contain isolated observations that lack an end-to-end lifecycle view. Further, novel paradigms like LLM-based test scripting, self-healing test frameworks, and hybrid approaches using multiple AI modalities remain to be adequately covered in current review papers.

This survey fills these gaps by providing a **systematic and lifecycle-focused overview of AI-based software test automation**. We cover 76 industry tools and peer-reviewed papers spanning an extensive array of AI approaches adopted at various phases of the software testing lifecycle. Our goal is to construct a structured taxonomy that bridges academic research with industrial practices, offering a consolidated and critical understanding of the field's current state, emerging trends, and research challenges.

B. RESEARCH QUESTIONS

To guide this systematic investigation, the following research questions (RQs) are defined:

RQ1: How are AI techniques utilized for automated test case generation?

RQ2: What AI-driven methods are applied for test repair, maintenance, and optimization in dynamic software environments?

RQ3: To what extent are AI-powered testing tools integrated into modern CI/CD workflows and DevOps pipelines?

RQ4: What are the current trends, challenges, and opportunities for hybrid AI strategies in intelligent software test automation?

C. CONTRIBUTIONS

The primary contributions of this survey are:

- 1) **Lifecycle-Based Taxonomy:** We introduce a holistic taxonomy that organizes AI-driven testing research by testing tasks, AI techniques, and data modalities.
- 2) **Comparative Analysis:** We compare existing research to previous surveys, highlighting open areas, trending topics, and differences in methodologies.
- 3) **Combination of Academic and Industry Knowledge:** We combine outcomes from both scholarly studies and practical tools, providing a complete perspective of AI-based test automation in real-world scenarios.
- 4) **Research Challenges and Research Roadmap:** We identify key research challenges like explainability, scalability, and solid CI/CD integration and propose a research agenda for future work.

The contributions align with observed adoption trajectories, in which organizations are piloting guardrailed LLM-assisted authoring [29], [30], integrating self-healing and repair into User Interface/Application Programming Interface (UI/API) suites [31], [32], and applying test-impact analysis to curtail redundant regressions [33]. These capabilities deliver defensible cost-of-quality gains—earlier fault detection and fewer unstable executions—while establishing governance hooks (explanations, traceability) required in regulated settings [34], [35]. Within this context, our systematic and empirical approach positions the survey as an exhaustive handbook for researchers, practitioners, and tool builders advancing AI-driven software testing within the broader AI4SE domain, thereby substantiating both its near-term applicability and market potential.

D. ROADMAP & READING GUIDE

Section II details our PRISMA-informed methodology and dataset. Section III introduces core concepts and terminology used later. Section IV presents the lifecycle-aligned taxonomy (tasks, techniques, data) and summary heatmaps. Section V compares our findings with prior surveys via a side-by-side matrix and lifecycle coverage plots. Section VI synthesizes challenges and a time-phased research roadmap. Section VII concludes with actionable recommendations and future directions.

II. METHODOLOGY

For an open, reproducible, and thorough review process, this study adheres to the PRISMA¹ (Preferred Reporting Items

¹<https://www.prisma-statement.org>

for Systematic Reviews and Meta-Analyses) guideline. The whole process was a multi-source literature search, stepwise screening, systematic data extraction, and cross-dimensional classification and analysis.

A. SEARCH STRATEGY

A systematic literature search was conducted across six reputable scholarly databases: **Scopus**, **IEEE Xplore**, **ACM Digital Library**, **SpringerLink**, **ScienceDirect**, and **Google Scholar**. The search strategy targeted peer-reviewed publications published between **2020 and 2025**, written in **English**, and explicitly focused on the application of AI techniques within the domain of software test automation. A representative query used in Scopus is shown below: TITLE-ABS-KEY (("artificial intelligence" OR "machine learning" OR "deep learning" OR "neural networks" OR "large language models" OR "reinforcement learning" OR "AI-driven testing" OR "AI-based test automation" OR "autonomous software testing" OR "test oracle generation" OR "self-healing test") AND ("software testing" OR "test automation" OR "automated software testing" OR "test case generation" OR "test suite optimization" OR "GUI testing" OR "Behavior-Driven Development") AND ("automated system" OR "intelligent system" OR "autonomous testing" OR "AI-based framework" OR "testing tool" OR "test bot"))

Equivalent expressions were tailored to the syntax of each platform. Google Scholar was used manually for identifying additional preprints and grey literature.

B. INCLUSION AND EXCLUSION CRITERIA

Inclusion Criteria:

- Peer-reviewed journal or conference publications;
- Published between 2020 and 2025;
- Written in English;
- Focused on the application of AI techniques in software testing;
- Reporting tools, frameworks, or empirical evaluation methods.

Exclusion Criteria:

- Non-peer-reviewed content;
- Posters, or abstracts only;
- Papers unrelated to software testing (e.g., general AI theory);
- Inaccessible full texts.

C. PROCESS OVERVIEW

This review is guided by a PRISMA-informed systematic iterative process with the objective of methodological consistency and completeness of themes as demonstrated by Figure 1. Screening was conducted via Microsoft Excel and Rayyan, a systematic review specialist software. Blinded inclusion/exclusion, tagging, and conflict resolution between reviewers were enabled in Rayyan. Agreement of the reviewers was approximated using Cohen's Kappa

coefficient, which was 0.81, meaning that there was considerable agreement among screeners. The overall process consists of the following stages:

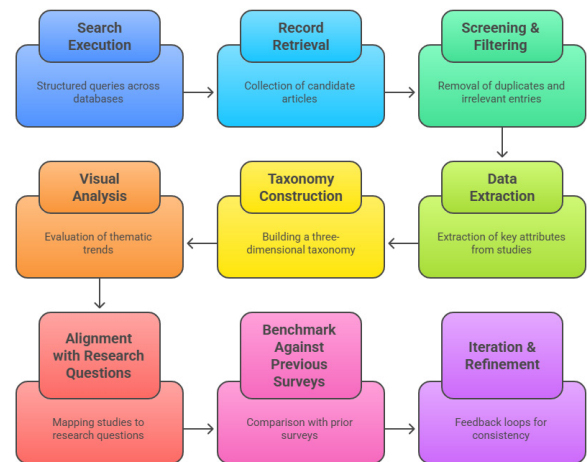


FIGURE 1. Systematic review process funnel.

- 1) **Search Execution:** Structured queries were issued across six major academic databases.
- 2) **Record Retrieval:** A total of **299 candidate articles** were collected based on title, abstract, and keyword matches.
- 3) **Screening & Filtering:** Duplicates and irrelevant entries were removed through multiple screening stages, including title check, abstract scan, and full-text assessment.
- 4) **Data Extraction:** Key attributes were extracted from the final 76 studies, including testing task, AI technique, data modality, tool names, and evaluation methods.
- 5) **Taxonomy Construction:** Extracted data was used to build a three-dimensional taxonomy classifying each paper by testing goal, AI technology, and data source.
- 6) **Visual Analysis:** Thematic trends and coverage were evaluated through radar charts, bar plots, heatmaps, and the PRISMA flow diagram.
- 7) **Alignment with RQs:** All the selected studies were aligned against one or more among the four given research questions (RQ1–RQ4) of the Introduction. This provided empirical alignment of evidence gathered to survey objectives.
- 8) **Benchmark Against Earlier Surveys:** Systematic procedure allowed for direct comparison to earlier surveys. In contrast to earlier reviews that neither provided lifecycle coverage nor addressed newer paradigms in AI, this survey offers a comprehensive and visually arranged overview.
- 9) **Iteration and Refinement:** Feedback loops were added between extraction and taxonomy steps to ensure

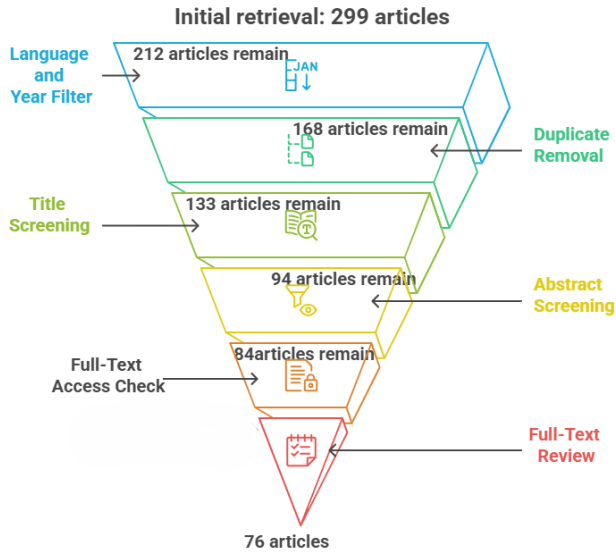


FIGURE 2. PRISMA flow diagram illustrating the study selection process.

consistency, completeness, and sufficient thematic tagging.

D. SCREENING AND SELECTION PROCESS

The screening process followed a staged structure with quantitative tracking at each phase:

- Initial retrieval: 299 articles
- After filtering by language and publication year: 212
- After removing duplicates: 168
- After title screening: 133
- After abstract screening: 94
- After full-text access check: 84 (10 articles excluded due to inaccessible full text)
- After full-text screening: 76 studies were included. Of the 84 full-text articles assessed, 8 were excluded according to our predefined criteria: 3 lacked empirical evaluation or implementation details, 3 did not clearly focus on AI-based software testing, and 2 provided insufficient methodological description to support classification or reproducibility.
- Final included studies: 76 (including 8 survey/literature review papers).

Figure 2 presents the PRISMA flow diagram summarizing the multi-step selection and filtering process, including the identification, eligibility, and inclusion phases.

E. DESCRIPTIVE ANALYSIS OF SELECTED STUDIES

To contextualize the final dataset, we conducted a descriptive analysis focusing on trends by publication year and document type. As shown in Figure 3, the number of relevant studies increases steadily from 2020, reaching a peak of 16 papers in both 2023 and 2024. By venue, 84% of the studies are conference papers and 16% are journal articles. We identified 8 survey/literature review papers, which we use as baseline

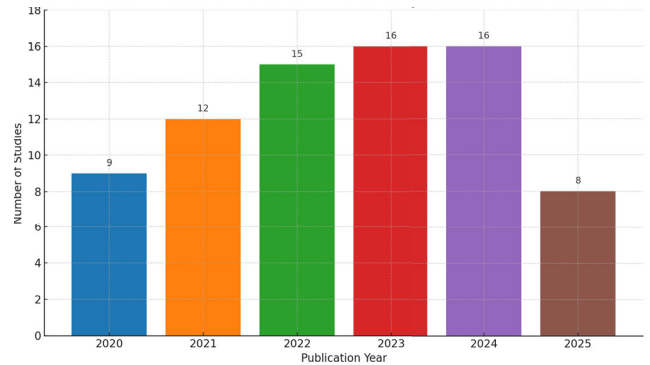


FIGURE 3. Annual distribution of selected studies.

references in our comparisons; the remaining 68 are original research contributions (e.g., method evaluations, tool proposals, and empirical studies). Notably, 75 of 76 studies propose or examine at least one automation framework, technique, or tool, underscoring the field's strong emphasis on practical implementation alongside conceptual advancement.

F. SUMMARY

This methodology, combining multisource retrieval, PRISMA filtering, structured extraction, iterative refinement, and multi-angle visual analysis, yields a rigorous, representative corpus of 76 papers on AI-driven software test automation. These results underpin the taxonomy, comparative review, and future research roadmap presented next.

III. BACKGROUND AND FUNDAMENTAL CONCEPTS

A. DEFINITION OF TEST AUTOMATION

Software test automation is the act of applying exacting scripts and tools to automate repetitive and time-consuming testing activities like test case execution, data preparation, environment configuration, and result validation. The aim is to minimize the need for manual intervention, increase uniformity, and speed up delivery cycles—especially in CI/CD pipelines [7], [8]. Even though test automation is widely used through tools like Selenium and JUnit, conventional test automation has well-known drawbacks. Automated scripts are bound to prove to be brittle, prone to error due to small changes in Application Programming Interfaces (APIs) or user interfaces, and with high maintenance overhead costs [7], [8].

Furthermore, traditional rule-based systems are ill-suited to contemporary architectures—microservices, event-driven, and real-time or data-intensive designs—because they encode fixed control logic and brittle selectors. Static scripts cannot exploit runtime telemetry to reprioritize execution paths, adjust time budgets, or react to context-dependent states, which leads to fragile and inefficient pipelines [9], [21], [36]. These restrictions are most important in the context of new areas like Internet of Things (IoT), autonomous control



FIGURE 4. SWOT analysis of AI-driven test automation.

systems, and safety-critical software, where response time and precision are the ultimate [23], [37].

Additionally, AI and ML-based system testing brings new challenges like the lack of test oracles, stochastic behavior, and non-determinism [38], that limit the efficacy of traditional rule-based methodology [11], [12]. In general, these challenges point out the limitations of static test automation and highlight the need for more intelligent, context-aware, hyper-adaptive testing approaches.

This call is well in alignment with the goals of the implementation, test, and maintenance phases of the AI4SE lifecycle where AI can bridge the gaps left behind by the traditional methods [11], [12]. Figure 4 presents a SWOT analysis of the key drivers and barriers to adopting AI-based testing in industry and academia.

The strategic outline represents the opportunities and limitations of deviating from traditional rule-based test automation to intelligent, adaptive approaches with the help of AI.

In terms of strengths, AI automation is very flexible and robust, particularly in the context of dynamic user interfaces and changing codebases where static test scripts tend to fail. The most significant weakness is the over-reliance on high-powered AI models, which require substantial training data, computing resources, and faith from testers and developers.

On the opportunity side, the growing demand for intelligent automation, precipitated by DevOps, CI/CD, and agile movements, provides a fertile landscape for AI-based solutions to grow in various industries. Nevertheless, there is a constant threat from sustained reliance on traditional rule-based models of automation that remain prevalent in most legacy systems and resource-constrained environments.

B. THE ROLE OF AI IN SOFTWARE TESTING

AI delivers strong solutions for the brittleness, rigidity, and high maintenance costs of traditional test automation. Unlike rule-based, static scripts, AI-based systems utilize intelligent methods, including ML, RL, DL, and semantic analysis, to autonomously generate, adapt, and optimize software [11], [12].

AI models may learn from various sources like source code, execution history, user activity patterns, and past test results to discover system behaviors and test paths that would otherwise remain undetected through traditional methods. For instance, LLMs such as GPT-4 and LLaMA may infer test logic from natural language specifications, API contracts, or user stories [5], [39]. Reinforcement learning agents can autonomously explore GUIs to uncover edge cases and latent states, as reported in recent mobile-testing studies [22], [40]. Recent work in mobile testing shows that reinforcement learning can learn GUI behavior and surface edge cases and hidden states [22], [40].

AI also facilitates sophisticated features such as historical execution pattern-based detection of flaky tests, semantic diffing-based repair of test scripts, and fault prediction model-based test prioritization [41], [42], [43]. In spite of these developments, there remain some limitations.

LLMs have no visual or UI context awareness when used within GUI-specific testing activities, for instance. RL agents need extensive training and have problems generalizing over application types [19]. Vision-based models, although enhanced in detecting layout shifts, are still vulnerable to UI rendering variation and localization issues [44].

To overcome these barriers, recent research promotes hybrid approaches that combine multiple AI paradigms, such as LLMs for test generation, RL for adaptive exploration, and CV for perceptual validation. These multi-modal strategies align with the broader goals of the AI4SE lifecycle, especially in the testing, deployment, and maintenance phases, where intelligent adaptability is crucial [11], [12].

C. MAIN APPLICATION DOMAINS OF AI-DRIVEN TESTING

AI techniques have been applied across a wide range of software testing. These domains relate to the various stages of the AI4SE lifecycle, such as implementation, testing, deployment, and maintenance.

1) TEST CASE GENERATION

Aligned with the implementation and early testing stages, test case generation is still the most sophisticated area in AI-based testing. Techniques such as LLMs and evolutionary algorithms are used to derive tests from API specifications, code, or user stories [5], [39]. Tools such as Pynquin and EvoSuite are examples of applying static and dynamic analysis for this purpose [24]. RL is also employed to explore execution paths or GUI interactions, making test creation more adaptive [21], [45].

2) TEST REPAIR AND OPTIMIZATION

Test repair aligns with the AI4SE maintenance phase of AI4SE, where evolving software systems require test scripts to be updated or optimized. RL and concolic methods are used for repairing faulty tests, eliminating redundancy, and prioritizing high-risk-of-failure scenarios [23], [41], [46],

TABLE 1. Conceptual alignment of taxonomy dimensions of development lifecycle with SDLC.

SDLC	Task-Based	Technology-Based	Data-Based
1. Requirements Analysis	(Emerging) Early test planning using LLMs	LLMs (prompt-based reasoning)	Specifications, user stories, API contracts
2. Code Implementation	Test generation from source code	LLMs, DL (code synthesis, Abstract Syntax Trees (AST) analysis)	Source code, bytecode, ASTs
3. Testing	Test generation, GUI testing, test repair	RL, CV, LLMs, DL	GUI screenshots, layout metadata, execution logs
4. Deployment	CI/CD integration and adaptive execution	ML (log modeling), RL (self-healing agents)	Execution logs, telemetry, pipeline traces
5. Maintenance & Evolution	Test repair, flaky test handling, CI/CD monitoring	RL, CV, DL (runtime adaptation)	Runtime logs, patch history, failure reports

[47]. Fault injection and mutation clustering techniques also support enhancing test efficacy [9], [48].

3) GUI AND VISUAL TESTING

GUI testing lies at the intersection of testing and user experience validation. RL agents with Computer Vision (CV) models engage with GUI components to simulate real user actions and detect layout inconsistencies [20], [40], [49]. Unlike older model-based tools, more recent frameworks accommodate dynamic GUI states and visual noise, which are essential for mobile and cross-platform applications.

4) CI/CD INTEGRATION AND RUNTIME ANALYSIS

These methods allow for the deployment and maintenance stages. AI infrastructure offers dynamic test choice, flaky test detection, and fault prediction based on logs and telemetry data.

Tools like Snowcat employ Graph Neural Networks (GNNs) for concurrency testing in Operating System (OS)-level systems [26], whereas ISTA and MindFI enable test scheduling and fault resilience in production environments [9], [50]. This real-time, feedback-based test model is a conceptual departure from static test execution to intelligent monitoring and adaptation [51].

D. COMMON AI-POWERED TESTING TOOLS

Numerous tools have emerged to support the domains discussed above. Rather than listing them independently, we categorize them into four functional clusters:

- **LLM-based Test Generators:** *Penguin* (Python + LLM) [5], [47], *EvoSuite* (Java + evolutionary algorithms) [24], *MUBot* (Android testing using GPT) [20]
- **RL/CV-based GUI Testers:** *UniRLTest* [22], [52], *OwlEye* [40], [53], *GUIRunner* [54]
- **Log/Runtime-Driven Tools:** *Snowcat* (GNN + kernel logs) [26], *ISTA* [50], *MindFI* [9]
- **Mutation and Robustness Tools:** *DeepCrime* (DL model mutation) [37], *AutoTestFixer* [55], *CrashDroid* (Android crash detection) [56]

While these tools show enormous developments in AI-driven testing, their reproducibility levels vary substantially. For example, *Penguin* is open-source and hosted on GitHub with complete access to code, test cases, and evaluation data.

In contrast, *GUIRunner* and *MUBot* are closed-source and have no implementation details available in public. Tools like *DeepCrime* and *MindFI* provide partial data sets or model configurations, but complete replication pipelines are not provided. These differences necessitate the utilization of standardized benchmarks and open-access repositories in order to promote empirical rigor and support future research validation. Figure 5 provides a complementary visual summary that illustrates how selected tools map to varying levels of functional maturity across the AI4SE testing landscape. As a Summary, these tools report overall trends in novel hybrid LLM+RL models, increasing integration with DevOps environments, and ongoing challenges in GUI robustness and explainability. While promising prototypes exist, only a few tools achieve full lifecycle coverage or production and scale deployment, highlighting opportunities for future integration frameworks [11], [17], [43].

IV. TAXONOMY OF EXISTING WORK

To ensure conceptual alignment with AI4SE, our taxonomy has been constructed with alignment with the most important stages of the AI-assisted SDLC, namely requirements analysis, code implementation, testing, deployment, and maintenance. Each of the taxonomy tasks, technologies, and data modalities aligns to at least one of these stages, setting our lifecycle-focused perspective. Table 1 presents a visual summary of this alignment.

This taxonomy results from a close reading of 76 peer-reviewed papers and presents a layered interpretation of current trends in research, technical limitations, and integration challenges. Below, we present each axis of the taxonomy, beginning with task-based classification of AI test approaches.

A. TASK-BASED TAXONOMY

The first dimension categorizes AI-based testing research by the primary functional objective of the work, whether it focuses on generating tests, repairing them, validating GUIs, or integrating testing into CI/CD workflows.

It makes it easier to identify research focus and areas of interest that are under-researched. These activities belong to some stages of the AI4SE life cycle: for example, test generation belongs to implementation and testing, and CI/CD

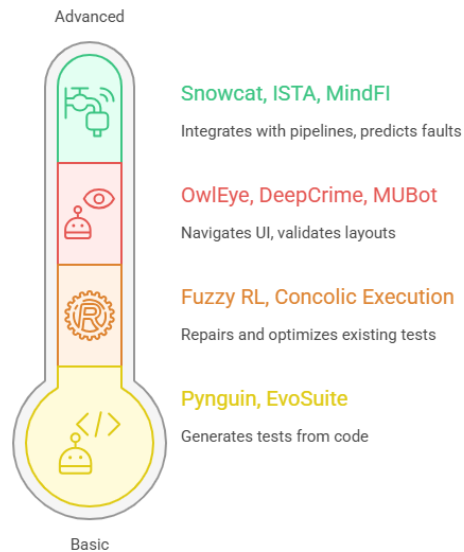


FIGURE 5. Functional maturity of representative AI-powered testing tools and techniques.

integration belongs to deployment and maintenance. Table 2 summarizes the distribution of the 76 reviewed articles in four main task categories.

TABLE 2. Distribution of papers by AI-driven testing tasks.

Task Category	counts	
GUI and Visual Testing	24	██████████
Test Case Generation (LLM, RL, CV)	26	██████████
Tool Evaluation & CI/CD Integration	16	██████████
Test Repair and Optimization	10	██████████

1) TEST CASE GENERATION

Test case generation remains the most extensively studied task among AI-driven testing methods. From the corpus under analysis, 26 papers specifically aim for this objective, typically employing LLMs, RL, or CV to generate test cases automatically from code, specifications, or natural language prompts.

Recent studies like [39], [43], [57], and [58] illustrate how LLMs like GPT-3.5, GPT-4, and LLaMA are being used more and more to synthesize tests from code snippets or user stories. For example, in [39], prompt-chaining and fine-tuning are employed to maximize the relevance and coverage of generated tests. Similarly, [57] benchmarks GPT-3.5-generated unit tests against traditional tools like EvoSuite, achieving comparable levels of performance based on mutation scores and human-readability.

Complementary to LLMs, RL techniques (e.g., Q-learning, Deep Q-Networks (DQNs)) are applied to manage test generation by state exploration and interaction modeling. Papers such as [21] and [45] show the application of RL to mobile app testing (UniRLTest, Fastbot2), where agents

autonomously explore the GUI state space and generate input sequences based on observed feedback and coverage metrics.

While less common, CV-based generation plays a key role in GUI-heavy systems. Tools like OwlEye and MUBot [20], [59] extract actionable UI elements from screen captures using object detection and segmentation, translating them into executable tests via RL-guided policies.

Despite promising results, challenges persist in structural test coherence, reproducibility, and convergence. For instance, [42] points out that temperature tuning in LLMs introduces randomness that undermines test determinism, while [60] notes that RL agents often fail to generalize across applications without extensive retraining.

Across these tools, a clear trend emerges. LLMs are increasingly replacing traditional heuristics in generating semantically meaningful test cases, while RL agents enable adaptive exploration in dynamic UI environments. However, despite promising results, challenges such as reproducibility, test determinism, and integration into CI pipelines remain consistent across tool implementations.

2) TEST REPAIR AND OPTIMIZATION

Test repair and optimization is a relatively uncharted but extremely significant domain in AI-based testing, with only 10 papers dealing with it specifically. The main goal here is to ensure the integrity and effectiveness of test suites as systems evolve.

Studies such as [24] and [61] explore mutation-based test repair, where candidate changes to test scripts are generated and ranked based on syntactic or semantic alignment with the evolving software [51]. Semantic repairs, such as those seen in [62], rely on behavioral equivalence to assess the validity of modified tests.

Other research, including [9] (MindFI) and [55] (AutoTest-Fixer), introduces self-healing frameworks that dynamically reconfigure or mutate failing tests at runtime. These systems utilize fault injection, reinforcement feedback, and repair cost models to provide coverage and stability in the case of interface or logic changes.

One of the most salient limitations in such studies is the lack of integration with real-world CI/CD environments. For example, [61] and [63] note that while theoretical repair strategies can guarantee correctness in isolated settings, their runtime overhead and poor pipeline coupling prevent widespread adoption.

3) GUI AND VISUAL TESTING

GUI and visual testing have emerged as a key research domain, represented by 24 papers in the dataset. The task here is to validate and interact with GUIs using AI-based agents or perceptual models, aiming to simulate user behavior and identify visual inconsistencies.

Vision-based frameworks such as OwlEye and DeepCrime [37], [40] utilize semantic image comparison techniques by employing object detection (e.g., You Only Look

Once version 5 (YOLOv5)) and saliency analysis to detect layout change and regressions. These tools outperform pixel-based comparison by reducing false positives and improving semantic relevance. Concurrently, RL-based GUI agents like UniRLTest and GUIRunner [21], [52], [54] model user flows by instantiating UI components with feedback-induced policies. These agents improve over static crawlers like TESTAR by adapting to dynamic interface structures.

Some of the hybrid approaches, such as those discussed in [19] and [64], integrate both CV and RL to enable robust cross-platform GUI validation. These models handle complex phenomena like component animation, screen resolution scaling, and localization, offering greater test resilience. However, challenges in standardization, oracle modeling, and agent transferability still exist. Papers like [65] and [66] inquire for more robust evaluation metrics and visual robustness benchmarks, which remain largely absent in the field.

GUI testing tools combine CV perception and RL navigation for simulating user interaction, but are plagued by layout sensitivity, nonstandard oracles, and weak cross-platform transferability—highlighting the demand for mutual benchmarks and adaptive perception layers.

4) TOOL EVALUATION AND CI/CD INTEGRATION

A growing number of papers (16 in total) examine how AI-based testing solutions integrate into modern CI/CD environments. This task category includes dynamic test scheduling, flaky test detection, runtime fault prediction, and pipeline feedback loops.

For instance, [26] (Snowcat) and [9] (MindFI) demonstrate how GNNs and reinforcement agents can be used to predict concurrency failures and reconfigure test flows. Reference [50] (ISTA) applies coverage-driven prioritization techniques to reduce test overhead in real-time systems.

Moreover, studies such as [27] and [67] explore adaptive testing policies that analyze commit-level changes and historical test logs to determine what subsets of tests to re-run, thereby minimizing execution time and increasing pipeline efficiency.

However, despite technical feasibility, practical adoption remains limited due to brittleness in script integration, lack of developer trust, and absence of visual dashboards or standardized APIs. Papers [37] and [43] highlight this deployment gap and emphasize the need for explainable AI and robust logging mechanisms in real-world DevOps workflows.

Taken together, these tools highlight a shift toward real-time, feedback-driven testing within CI/CD environments. While conceptually mature, their practical adoption is hindered by issues of brittleness, insufficient logging standards, and trust gaps. Common trends point to the value of explainable AI, fault tolerance, and integration APIs in future CI/CD-focused frameworks. While test generation tools dominate current research, CI/CD-integrated test oracles and

autonomous feedback mechanisms remain underexplored, representing a critical opportunity for industrial AI testing pipelines.

5) SUMMARY AND INSIGHTS

The task-based analysis reveals distinct strengths and weaknesses across categories:

- **Test generation** is the most active research area, driven by advances in LLMs and RL, but suffers from reproducibility and GUI consistency issues.
- **Test repair**, while essential, remains academically underrepresented and weakly integrated into CI/CD systems.
- **GUI testing** benefits from CV and RL but lacks standard oracles and suffers from platform variance.
- **Tool evaluation** in CI/CD shows maturity in conceptual models but faces operational barriers in deployment and validation.

Few studies bridge multiple testing tasks—e.g., coupling test generation with repair or GUI validation—underscoring the need for hybrid, end-to-end frameworks across the life-cycle. Beyond tool-specific features, our taxonomy surfaces broader trends: the convergence of LLMs with RL, increasing use of perception-driven (CV) techniques in GUI testing, and growing reliance on real-time CI/CD feedback.

B. TECHNOLOGY-BASED TAXONOMY

The second axis of our taxonomy organizes the reviewed literature based on the *AI techniques* employed to enable intelligent testing capabilities. Unlike task-based classification, which focuses on the “what” (i.e., functional testing goals), this technology-based view addresses the “how,” highlighting the specific ML models, training strategies, and adaptation mechanisms used to achieve those goals. Each AI technique in this classification supports specific lifecycle stages. For example, LLMs are prominent in implementation and test generation, while GUI testing and maintenance commonly combine LLM and DL with RL- and CV-based components, reflecting the need for both interaction policy learning and visual perception.

The categorization includes five major technique families observed across the 76 papers: (i) Classical ML, (ii) DL, (iii) RL, (iv) LLMs, and (v) CV. While these technologies are often used in combination, we group each paper according to its dominant implementation focus.

1) CLASSICAL MACHINE LEARNING

Traditional ML algorithms such as Support Vector Machines (SVM), Decision Trees, and clustering models are used in tasks like flaky test prediction, risk-based prioritization, and fault detection. For instance, studies such as [63], [67], and [68] apply statistical classifiers trained on historical test execution data to predict unstable test paths or prioritize test selection.

Although classical ML offers interpretability and low inference cost—ideal for CI/CD environments—it suffers from limited expressiveness. As noted in [28] and [69], these models cannot capture complex temporal or structural relationships in software behavior and require substantial manual feature engineering. However, ML techniques have demonstrated value in predictive modeling and Monte Carlo-based simulation across other domains such as hardware-level reliability studies [70].

2) DEEP LEARNING

DL models such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and GNNs are widely used in structural analysis, visual recognition, and concurrency fault prediction. For example, DeepCrime [37] applies CNNs for fault injection analysis, while [71] uses Long Short-Term Memory (LSTM)-based models to mine log sequences. GNNs are used in Snowcat [26] to analyze kernel concurrency.

Despite their power, DL models are data-hungry, lack explainability [72], and raise concerns in regulated domains. Their effectiveness depends heavily on access to large, annotated datasets [73].

3) REINFORCEMENT LEARNING

RL is applied in test sequence optimization and autonomous GUI navigation. UniRLTest [21] and GUIRunner [54] employ DQNs to interact with Android UIs, guided by test coverage rewards. Other studies like [45] and [60] explore Q-learning and policy refinement techniques for both GUI and repair tasks. However, RL agents require extensive training, have limited generalization, and are hard to reuse across software systems [74].

4) LARGE LANGUAGE MODELS

LLMs such as GPT-4, GPT-3.5, LLaMA, and PaLM are increasingly used for natural-language test generation. Studies [5], [39], [43] demonstrate that these models produce high-quality, readable test cases from API contracts or user stories. Techniques like prompt chaining [39] and domain-specific tuning [72] improve test relevance.

Nevertheless, LLMs introduce reproducibility issues [42], struggle with GUI state alignment [57], and lack execution determinism without extensive post-processing.

5) COMPUTER VISION

CV is used in GUI-based validation and layout regression analysis. OwlEye and DeepCrime [37], [40] apply semantic image diffing using YOLOv5 and saliency maps. CV enables perception-based testing, avoiding brittle pixel comparisons.

Yet, as shown in [54] and [66], CV tools are sensitive to rendering variations, localization, and lack standard metrics for visual robustness or correctness validation.

6) SUMMARY AND INSIGHTS

Technology-based analysis highlights the distinctive advantages and limitations of different AI approaches in software test automation.

- **Classical ML** is efficient and interpretable, but limited to simple predictive tasks.
- **DL** is powerful for perception and trace modeling but demands large datasets and lacks transparency.
- **RL** enables adaptation and autonomy but suffers from poor generalization and long training cycles.
- **LLMs** are flexible and effective for text-based automation but lack precise control.
- **CV** enhances GUI testing but is fragile under layout shifts and lacks oracle standards.

These findings support the trend toward hybrid architectures that combine the strengths of different AI models, such as LLM-RL or DL-CV pipelines for robust, multi-modal test automation. These results suggest a natural convergence of AI techniques based on their alignment with specific testing challenges. For example, LLMs can consume in interpreting natural language specifications and generating structured test logic, making them suitable for early-phase test design.

Conversely, RL and CV approaches are predominantly effective in GUI-intensive applications where interaction modeling and visual feedback are essential. DL and GNN-based methods increasingly serve log analysis and fault prediction tasks in CI/CD settings. As the field matures, hybrid pipelines—e.g., LLM+RL or DL+CV—are emerging to bridge textual intent, execution context, and interface behavior.

This trend reinforces the shift from isolated AI technique usage to multi-modal, lifecycle-aware architectures in software test automation.

C. DATA-BASED TAXONOMY

The third dimension of our taxonomy classifies AI-powered software testing research by the *data modalities* to which learning or reasoning is conducted. Contrary to task- or technology-focused views, with emphasis on goal functional objectives and implementation approaches, this classification emphasizes the types and sources of information used to build, train, or guide AI testing systems. The data modalities discussed here are strongly correlated to some AI4SE phases. For example, source code is at the heart of test generation and implementation, while runtime logs and GUI layouts support testing, deployment, and maintenance.

The knowledge of data modalities is important because the effectiveness, generalizability, and performance of AI methods frequently rely more on the data structure and quality rather than the model itself. Our work settles the four most prominent data types in the state of the art as: (i) Source Code and Bytecode, (ii) GUI Layouts and Screenshots, (iii) Execution Logs and Runtime Traces, and (iv) Synthetic vs. Real-World Test Data.

1) SOURCE CODE AND BYTECODE

A large subset of the reviewed papers relies on static code representations—either in the form of source code (e.g., Java, Python) or intermediate representations such as Abstract Syntax Trees or bytecode. These data are used primarily for unit test generation, mutation analysis, and symbolic modeling. For example, papers [24], [39], [57] employ language models trained on open-source repositories to derive syntactic and semantic test logic.

In addition, some works such as [42] and [44] utilize GNNs over control flow graphs or program dependency graphs extracted from source code to detect test coverage gaps or predict flaky tests. Although code-based input is deterministic and easy to acquire, it requires advanced parsing and transformation pipelines to extract meaningful features, especially for polyglot or obfuscated code bases. Moreover, code alone often lacks behavioral context, requiring augmentation with dynamic data.

2) GUI LAYOUTS AND SCREENSHOTS

As GUI-based applications dominate modern software systems, many tools rely on visual data such as rendered screenshots, Document Object Model (DOM) trees, or component metadata. Studies such as [19], [40], and [75] use screenshots to detect layout shifts, identify clickable elements, or map transitions between interface states.

These data are typically processed using CV techniques like object detection, image segmentation, or saliency mapping. For instance, OwIEye [40] uses YOLOv5 to localize UI widgets, while GUIRunner [54] builds interaction graphs from bounding box metadata.

While screenshots offer realistic testing inputs, they are inherently fragile. Screen resolution, font rendering, theming, and localization can all introduce variance. As noted in [66], there is a lack of standardized robustness metrics or normalization strategies to handle such variation.

3) EXECUTION LOGS AND RUNTIME TRACES

Several papers use execution traces, runtime telemetry, and debug logs to guide test prioritization or failure prediction. For example, [50] and [74] analyze runtime logs to detect flaky tests, while [27] applies statistical modeling to test reruns based on log stability.

Log-based data provides temporal and behavioral information that static analysis cannot capture. Papers such as [71] and [76] employ RNNs and anomaly detection to learn patterns of execution leading to failures or performance bottlenecks.

Despite their utility, logs are often noisy, inconsistent across platforms, and hard to annotate. Furthermore, as [61] and [68] note, logs may omit crucial events due to sampling or logging configuration errors, limiting their reliability.

4) SYNTHETIC VS REAL-WORLD TEST DATA

Another key dimension is whether the input data is synthetically generated (e.g., via fuzzing, LLM-based generation, or mutation) or collected from real software systems. Synthetic data is attractive for its controllability and scalability, seen in works such as [52] and [77], where LLMs generate test cases for API functions and UI flows.

Conversely, real-world test data, drawn from production environments or community benchmarks like Themis or GitHub repos, offer higher ecological validity. Studies such as [45] and [78] build datasets from Android apps and open-source projects, enabling empirical evaluation under realistic conditions.

However, real data often requires anonymization, labeling, and version tracking, which increases overhead. As [65] and [79] highlight, there is also a trade-off between representativeness and re-usability: what works well for one domain may not generalize.

5) SUMMARY AND INSIGHTS

The analysis of input data sources reveals several key insights:

- **Source code and bytecode** provide rich structural information but lack behavioral dynamics.
- **Screenshots and layout data** enhance realism but are sensitive to environmental changes.
- **Execution logs and traces** capture runtime behavior, but are difficult to standardize and annotate.
- **Synthetic data** scales easily but may not reflect real-world complexity; **real-world data** offers authenticity at the cost of curation effort.

Future AI-based testing systems must account for these trade-offs by adopting hybrid data strategies, employing dynamic adaptation layers, and developing robust benchmarks for cross-modal input evaluation.

Analyzing the reviewed tools and techniques reveals distinct patterns in data usage. Source code and ASTs remain foundational for static analysis and test generation tasks, particularly those involving LLMs and symbolic execution. GUI layouts and screenshots prominently support RL- and CV-based test agents that simulate human interactions, and are frequently paired with LLM/DL components. Meanwhile, runtime logs and telemetry data are central in ML- and DL-driven fault prediction and CI/CD optimization. However, across all data types, two overarching challenges emerge: (i) the lack of standardized, domain-agnostic benchmarks, and (ii) the difficulty of integrating diverse data modalities into cohesive AI pipelines. Addressing these issues is crucial for building scalable, generalizable AI testing frameworks that adapt across software domains and lifecycle phases.

6) CHALLENGES IN BENCHMARK DATASETS AND DATA QUALITY

One of the largest limitations throughout the surveyed studies is the absence of standard, publicly available benchmark

datasets upon which to compare AI-based software testing methods. Although datasets such as CodeSearchNet or HumanEval provide useful resources for code-based test generation [80], [81], there are no comparable wide datasets for GUI testing, runtime trace analysis, or CI/CD pipeline testing. This fragmentation hinders reproducibility and impairs the objective comparison of tools and techniques across studies.

Many experimental studies use internally simulated or synthetic data, which generally is not publicly released because of confidentiality, privacy, or an inability to make any curation attempt [82]. This makes it difficult to validate claims or generalize findings to diverse software ecosystems. For example, GUI-focused tools often evaluate performance on a limited set of Android or desktop applications, with no standardized layout representations, interaction logs, or oracle annotations. Moreover, execution traces and runtime telemetry data, critical for debugging, flaky test detection, and test optimization, are not standardized in most studies [83]. Noise, sampling bias, and inconsistent logging practices make it difficult to train robust models or compare approaches fairly [80].

Solving these problems entails the creation of realistic and diversified benchmark datasets that cover different data modalities (source code, UI traces, logs), include ground truth annotations, and offer reproducible test oracles. Creating such benchmarks would add significantly to empirical rigor, enable reproducibility, and speed up the development of generalizable, lifecycle-aware AI testing frameworks.

D. TAXONOMY SUMMARY

In this section, we introduced a comprehensive three-dimensional taxonomy framework, systematically classifying existing research on AI-driven software testing based on testing tasks, AI techniques, and data modalities. This classification is derived from a rigorous analysis of 76 peer-reviewed studies, providing an organized and insightful overview of the current state of research.

To reinforce the practical relevance of our taxonomy, each axis, including tasks, technologies, and data, is conceptually mapped to key stages of the AI4SE for SDLC process, as detailed in Table 1. Three heatmaps presented at the end of this section effectively highlight the intricate relationships among these three critical dimensions. Unless noted otherwise, all counts refer to assignment frequencies rather than unique papers; a single paper may contribute multiple assignments.

Figure 6 illustrates the relationship between software testing tasks and AI techniques.

Notably, CV and RL techniques exhibit substantial utilization in GUI and visual testing tasks (9 and 7 papers, respectively), and DL is comparably prominent here as well (7 papers), reflecting their strengths in handling dynamic user interactions and visual complexity. LLMs are predominantly used for test case generation (16 papers), with RL (8) and DL (7) playing secondary roles. Meanwhile, ML and DL

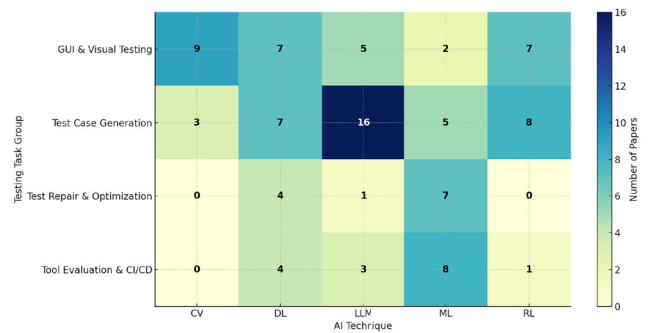


FIGURE 6. Heatmap showing AI technique usage across grouped testing categories.

are relatively prominent in tasks related to test repair and optimization (ML = 7, DL = 4) and integration within CI/CD environments (ML = 8, DL = 4), whereas RL appears only sporadically outside GUI/visual and generation tasks (CI/CD = 1; Repair = 0).

Figure 7 explores the interplay between AI techniques and data modalities. As depicted, LLMs rely primarily on GUI layouts and source code / API artifacts (23 and 20 papers, respectively), demonstrating their efficiency in the synthesis of text- and UI-driven test scenarios. RL and CV techniques are mostly associated with graphical and visual modalities such as GUI layouts and screenshots (16 and 12 papers), emphasizing their applicability to interactive and visual-centric testing scenarios.

Execution logs and telemetry are widely used - most prominently by ML (12) and then by LLM / RL / DL (7/7/6) - highlighting the importance of runtime information for behavior monitoring and failure prediction. In this context, LLM-based systems often use code repositories and API contracts (e.g., GitHub projects, interface specs) as well as extracted UI hierarchies; RL and CV tools are commonly evaluated on mobile apps and UI-centric datasets (Android apps, emulator screenshots).

Synthetic data appears but remains modest across techniques (LLM = 6, RL = 5, ML = 4, CV = 2, DL = 2), with some studies generating inputs via LLM prompting or fuzzing, while others rely on real-world logs and telemetry from production systems.

Figure 8 links testing tasks to the data modalities they use, completing the triad alongside the prior heatmap views. As expected, GUI and Visual Testing are dominated by GUI layouts (23 assignments), with non-trivial use of source code (11) and some execution logs (5). Test Case Generation frequently pairs GUI artifacts (26) with source code/API specifications (19), and also leverages synthetic data in several studies (9).

Test Repair and Optimization relies primarily on execution logs/runtime traces (6) and some GUI artifacts (6), with limited use of source code (2) and synthetic data (1). Tool Evaluation and CI/CD show a more balanced mix—source code (9), GUI (8), and logs (5). The patterns align with

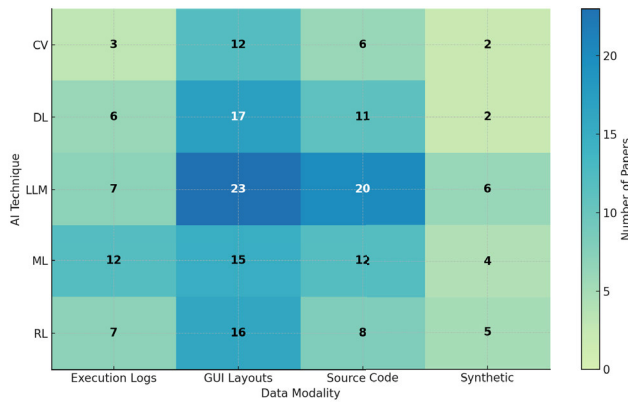


FIGURE 7. Heatmap depicting the relationship between AI techniques and data modalities.

the lifecycle-oriented taxonomy and earlier heatmaps. GUI-centric tasks are inherently visual, generation pairs code with GUI artifacts, and runtime logs are central to maintenance and CI/CD workflows.

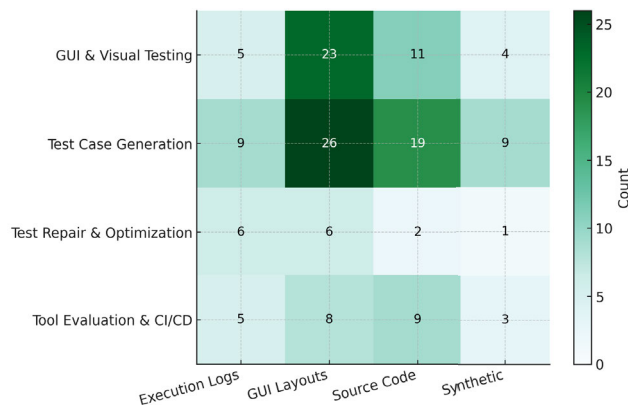


FIGURE 8. Heatmap showing data modalities usage across grouped testing categories.

Overall, the outcomes from this taxonomy underscore clear research patterns in the literature: LLMs are most prominent in generating test scenarios, reflecting their strengths in code and language understanding; GUI-centric testing is largely driven by CV and RL, with DL comparably present and LLMs still visible in hybrid setups; and log-oriented analyses align more with ML (with DL also represented), indicating a growing emphasis on robustness and reliability under realistic runtime conditions. In parallel, the use of synthetic data is concentrated in test generation, with comparatively limited adoption in repair and CI/CD contexts, pointing to a promising avenue for deeper exploration.

Future work should prioritize unified, multimodal pipelines that fuse logs, GUI traces, and source code. Likewise, hybrid AI strategies, such as coupling LLMs with RL agents, or combining CV/DL with symbolic feedback oracles, can address current pain points around determinism, robustness, and GUI sensitivity. Developing

benchmark datasets that blend synthetic and real-world samples, especially beyond generation, will be essential for reproducible and generalizable evaluation.

E. FINDINGS AT A GLANCE

- **Lifecycle coverage.** Evidence concentrates on *test generation/authoring* and *GUI/visual testing*; *test repair/maintenance* and *CI/CD operationalization* remain underrepresented.
- **Technique mix.** LLMs dominate NL for test authoring; DL/CV underpin visual oracles; classical ML supports prediction tasks (e.g., failure risk); RL shows promise for prioritization/optimization but with limited evaluations.
- **Data modalities.** Code, GUI images, and logs are the prevalent modalities; *multi-modal* pipelines (code+GUI+telemetry) are emerging but are rarely evaluated end-to-end.
- **Evaluation practice.** The field lacks *standardized benchmarks*, *shared baselines*, and *reproducible protocols*; heterogeneous setups constrain cross-paper comparability and favor directional (not absolute) conclusions.
- **CI/CD integration.** Practical integration is uneven. *Flaky-test triage* and *test-impact analysis* appear in pilots, but scalability, determinism, and governance (logging, versioning, audit) are frequent pain points.
- **Explainability (XAI).** Model-level attributions are sporadically reported; *artifact-level* rationales (for generated steps, oracles, or repairs) are rare and rarely logged with build metadata—limiting trust, reviewability, and audit.
- **Hybrid strategies.** Compositions of LLMs+RL+CV are of growing interest; however, current architectures are early-stage and seldom validated on realistic, longitudinal workflows.
- **Adoption signals.** Industrial interest is strongest in web/mobile/API contexts; key blockers include *maintenance cost* (locator churn), *scalability* (data/compute), and *policy/assurance* (privacy, compliance).

V. COMPARATIVE ANALYSIS OF TECHNIQUES

In recent years, the rapid growth of AI applications in software testing has led to the emergence of multiple surveys in this area. For instance, [27] explored the general opportunities of using AI in software testing, tool-oriented and end-to-end (E2E), but provided limited coverage regarding the full test lifecycle phases. Reference [28] focused more on AI-based defect prediction frameworks, with less emphasis on automation tasks like GUI or visual testing. Reference [15] analyzed AI-assisted test automation via grey literature with taxonomies and tool trends, but not lifecycle-aligned and without academic empirical validation.

In addition, [84] cataloged AI techniques for software test automation but only mapped them roughly to testing activities without systematic breakdowns or comprehensive tool evaluation. Reference [85] provided a broad overview of

AI usage in software testing using ML and DL, but with only 20 papers and no structured mapping to lifecycle activities. Reference [86] mainly highlighted the benefits of AI in test automation from a high-level industrial perspective but did not offer an in-depth taxonomy or coverage of specific automation phases. Reference [87] conducted a valuable tool-oriented systematic review and empirical evaluation of AI-powered test automation tools, though its focus remained primarily on self-healing and visual validation features. Finally, [88] emphasized ML for test generation across testing types but focused mostly on algorithms and lacked a lifecycle-based perspective or tool coverage.

Despite these valuable contributions, a detailed analysis reveals that significant gaps still exist across these studies, which our current survey directly addresses.

A. MAIN SHORTCOMINGS OF PREVIOUS SURVEYS

A closer look at prior surveys reveals several critical shortcomings that limit their scope and practical impact.

- **Incomplete coverage:** Key phases—test planning, test design, test authoring, test execution, test maintenance, and test closure—remain underrepresented ([27], [85], [88]).
- **Lack of a systematic taxonomy:** Prior surveys often emphasize tool-level descriptions or generic problem/solution spaces without a clear mapping to lifecycle activities ([28]).
- **Limited technical depth in CV-based visual validation for GUI testing:** Prior work emphasizes tool capabilities over underlying computer-vision methods and evaluation details ([15], [86], [87]).
- **Limited coverage of Natural Language Processing (NLP)-based test scripting:** Prior work underexplores its potential to democratize test automation by enabling non-programmers to generate executable test cases ([85], [88]).
- **Lack of a consolidated, lifecycle-mapped table:** Prior surveys rarely present a single matrix that explicitly identifies overlaps and gaps across the testing lifecycle ([27], [28], [85]).
- **Limited incorporation of empirical validation:** Few surveys ground findings in evaluations on real-world open-source projects (notably missing in [27], [86], [88]).

B. NOVEL CONTRIBUTIONS OF OUR SURVEY

Our survey distinguishes itself by:

- **Providing a structured, lifecycle-based taxonomy** that spans the entire software testing process, integrating planning, authoring, execution, repair, and maintenance stages with AI techniques.
- **Deep analysis of GUI and visual testing**, incorporating approaches based on CV, and comparing tool capabilities for visual inspection and layout robustness.

- **Highlighting the role of NLP-based scripting** as an enabler for low-code/no-code test automation, which was overlooked in earlier surveys.
- **Synthesizing and positioning LLMs and RL within a lifecycle-aligned taxonomy**, capturing their role in modern test generation, prioritization, and autonomous repair.
- **Conducting a comprehensive comparative study**, using explicit lifecycle-based mapping and domain-aligned visualization, to clarify overlaps and distinguish gaps in prior work.
- **Grounding conclusions in empirical evidence**, including open-source datasets, benchmarks, and tools, to assess real-world applicability.

C. VISUAL COMPARATIVE ANALYSIS

To better illustrate the distinctions between our survey and previous studies, we have developed detailed visual comparisons.

- **Bar Chart:** Figure 9 contrasts, for each survey, which test-lifecycle areas are covered (stacked bars). The comparison reveals sparse coverage of GUI testing and NLP-based scripting across prior surveys, while test closure is not explicitly addressed and thus not included in the plot. Our survey is the only one spanning all seven areas (planning, design, authoring, execution, maintenance, GUI, and NLP-based scripting).
- **Comparative Table:** Table 3 summarizes each related survey's scope, focus area, answer to our RQs, tools analyzed, and academic/industrial perspective, allowing a direct side-by-side comparison of contributions, gaps, and lifecycle coverage.

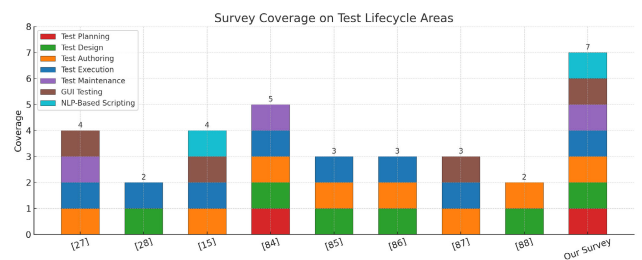


FIGURE 9. Number of surveys covering each lifecycle phase.

VI. FUTURE DIRECTIONS AND RESEARCH ROADMAP

While AI-driven test automation has made substantial progress, several technical challenges and emerging research opportunities continue to shape its evolution.

To provide a strategic vision for future work, we reduce current gaps and ideal trends into a layered research roadmap structured with short-, medium-, and long-term horizons.

A. TECHNICAL CHALLENGES

Despite growing adoption, significant technical limitations constrain the scalability and trustworthiness of AI-powered

TABLE 3. Comparison of existing surveys (Does each survey answer our RQs?).

Survey	Scope	Focus Area	Answers our RQs?	Tools Analyzed	Perspective
[27]	AI-augmented End-to-End (E2E) Test Automation	Tool-oriented view (self-healing, visual, NLP, ...)	No	Several representative tools (8)	Mixed
[28]	AI for Defect Prediction	Defect prediction models (Machine Intelligence (MI) MI/ML-based)	No	Limited / conceptual	Academic
[15]	Grey Literature Synthesis	Industry insights, taxonomies, tool trends	Yes (RQ1, RQ4)	100+ tools cataloged	Industrial (grey literature)
[84]	AI in Test Automation	Mapping AI techniques to testing activities	Yes (RQ1)	14 tools; 34 papers	Academic
[85]	General AI in Testing	ML/DL-based software testing tasks; what can be automated	Yes (RQ1)	20 studies (not tool-centric)	Academic
[86]	High-Level AI Benefits	Advocacy/vision; mentions tools	No	Mentioned, not systematically reviewed	Industrial
[87]	Tool-Centric Review	Features/limitations of AI-testing tools	No	55 tools; 2 demo-tested	Mixed
[88]	ML for Test Generation	ML-based test generation techniques	Yes (RQ1)	Conceptual; no tool audit	Academic
Our Survey	Full Lifecycle + AI + Empirical Synthesis	Lifecycle +LLM/RL/DL/ML/CV, Generation/GUI/Repair/CI/CD	Yes (All RQs)	76 studies + tools	Academic + Industrial

testing tools. Many solutions remain difficult to deploy—despite sound theoretical underpinnings—due to brittle script/pipeline integration, insufficient documentation and operational guidance, and limited explainability, particularly in safety-critical domains where trust and auditability are non-negotiable. Addressing these gaps is essential for real-world impact.

- **Explainability:** Most AI models, especially LLMs and deep neural networks, are inherently opaque models, eliminating interpretability, stakeholder trust, and debugging transparency. Future work should prioritize robust interpretable ML and model-agnostic explanation methods, including complex feature-attribution approaches (e.g., SHAP, LIME), as well as counterfactual and example-based explanations. Wherever feasible, explanations must be thoroughly integrated into test artifacts (steps, oracles, repairs) and logged within CI/CD for auditability [9], [72].
- **Scalability:** Current methods can struggle at large scale and on complex systems due to data and computational overheads [26]. Advancing scalability will require research on distributed/parallel architectures, hybrid AI-heuristic pipelines (to limit search and cost), and model-optimization strategies (e.g., compression, distillation, caching, and hardware-aware inference) [23].
- **Benchmarking:** The discipline is bedeviled by a lack of standard datasets, evaluation metrics, and reproducible protocols [89], [90]. Establishing benchmark suites across modalities (code, GUI, logs) is essential for fair comparison and empirical rigor, complemented by shared baselines, reporting checklists, and artifact repositories to ensure replicability and sustained comparability [91].

B. EMERGING RESEARCH OPPORTUNITIES

We identify two fast-growing research trajectories with transformative potential.

- **LLM-based Testing:** LLMs such as GPT-4 and LLaMA are promising for test case generation, bug explanation, and requirement-to-test translation. Prompt engineering optimization, domain-specific testing fine-tuning, and strong evaluation frameworks are open problems awaiting solution [17], [20], [43].
- **RL-based Optimization:** RL offers adaptive strategies for test scheduling, selection, and GUI navigation. Reward shaping, generalization to unseen UIs, and scalable integration into CI/CD pipelines are avenues of future research to pursue [21], [50], [60].

C. STRATEGIC RESEARCH ROADMAP

To outline a roadmap for the future, we suggest a three-phase research roadmap in Table 4 that connects capabilities to the maturity of AI4SE development pipelines.

This roadmap helps position emerging techniques within a broader AI4SE lifecycle, encouraging a shift from isolated tooling to integrated, resilient, and explainable AI testing infrastructures. In addition to technical innovation, future research should consider:

- **Integration with DevOps:** Seamlessly embed AI components into software delivery pipelines.
- **Hybrid AI Models:** Combine symbolic, statistical, and perceptual AI to overcome individual model limitations.
- **Cross-Domain Generalization:** Train models that generalize across programming languages, UI libraries, and deployment platforms.
- **Ethical and Legal Considerations:** Address fairness, bias, and accountability in autonomous test agents.

TABLE 4. Research roadmap for AI-driven software testing.

Horizon	Research Focus	Representative Challenges and Goals
Near-Term (1–2 years)	Hybrid AI pipelines	Combine LLMs + RL + CV for test generation and GUI validation; improve adaptability [20], [43]
Mid-Term (3–5 years)	Explainability and DevOps integration	Develop interpretable AI, robust pipelines, and benchmark datasets; integrate with CI/CD [9], [72]
Long-Term (5+ years)	Trustworthy, domain-generalizable systems	Ensure fairness, safety, adaptability in high-stakes applications (e.g., autonomous systems) [11], [44]

- **User-Centric Testing:** Incorporate end-user behavior and feedback loops into test planning and prioritization.

While not a prescriptive standard and acknowledging contextual variability across organizations, by aligning short-term feasibility with long-term vision, this framework is intended to steer future work toward developing trustworthy, scalable, and comprehensible AI systems for software testing.

Our inclusion criteria require *peer-reviewed* method descriptions and empirical evaluation; accordingly, community-driven open-source tools lacking peer-reviewed documentation or reproducible studies were excluded from the corpus. We acknowledge that this constraint may bias the synthesis toward more *visible* published methods.

VII. CONCLUSION

This survey provides a wide-ranging and in-depth overview of AI-driven software test automation within the broader AI4SE landscape. By analyzing 76 industrial tools and peer-reviewed articles, we illustrated how AI techniques—ML, DL, LLMs, and RL—are revolutionizing the software testing life cycle. We introduced a life cycle-based taxonomy that organizes the state of the art by testing tasks, AI methods, and data modalities, thereby synthesizing recent trends, revealing gaps, and outlining future possibilities.

Our comparative analysis reveals solid progress in test case generation and GUI/visual testing, while pointing to persistent gaps in test repair and optimization, scalability within CI/CD pipelines, and explainability of AI-driven decisions. We further observe an acute shortage of standardized benchmarks and datasets, which inhibits robust empirical validation and limits real-world applicability.

We identified promising research avenues, including LLM-based test scripting, RL-driven test suite optimization, and hybrid strategies that leverage multi-modal data. Advancing these directions is essential to address current limitations in generalization, robustness, and developer trust in AI-assisted testing.

A. FINDINGS BY RESEARCH QUESTION

RQ1: How are AI techniques utilized for automated test case generation? LLMs are currently the dominant approach for test generation and are often competitive with,

or better than, heuristic baselines; however, *reproducibility* and *determinism* remain open concerns.

RQ2: What AI-driven methods are applied for test repair, maintenance, and optimization in dynamic software environments? While there is preliminary progress in test *repair* and *optimization*—notably via RL and concolic techniques—these methods are still immature and weakly integrated into CI/CD workflows.

RQ3: To what extent are AI-powered testing tools integrated into modern CI/CD workflows and DevOps pipelines? Practical deployment in industrial DevOps remains limited, with *scalability* and *trustworthiness* as primary obstacles.

RQ4: What are the current trends, challenges, and opportunities for hybrid AI strategies in intelligent software test automation? Interest in *hybrid* approaches that combine LLMs, RL, and CV is growing, yet such architectures are nascent and rarely evaluated end-to-end.

B. ACTIONABLE RECOMMENDATIONS

In light of these findings, we recommend prioritizing:

- **DevOps integration:** Seamless incorporation of AI testing services into CI/CD (e.g., test-impact analysis, flaky-test quarantine, self-healing selectors) with audit trails and build metadata.
- **Explainability and governance:** XAI techniques tailored to testing artifacts (test intent, oracles, repair rationale) and policy hooks (model/prompt versioning, decision logs) to improve trust and compliance.
- **Benchmarks and datasets:** Open, standardized, and regularly updated suites (including GUI/API churn, flaky phenomena) to enable reproducible research and fair comparison.
- **Hybrid AI frameworks:** Compositions of LLMs, RL, DL, CV, and symbolic reasoning for robust, end-to-end automation with real-world constraints.
- **Cross-domain generalization:** Methods that preserve performance across programming languages, UI frameworks, and application domains, with focused stress tests for maintenance and drift.

C. ETHICS AND RISK

Future work should also address ethical and safety considerations—LLM hallucination, bias amplification,

privacy, and accountability in semi- or fully autonomous testing pipelines—especially where AI tooling influences release decisions.

By addressing these challenges, the field can progress toward *strong, scalable, and reliable* AI-driven testing that narrows the gap between academic innovation and industrial adoption. We hope this survey serves as a foundational reference for researchers, practitioners, and tool developers committed to advancing intelligent software testing in the evolving AI4SE ecosystem.

REFERENCES

- [1] M. Parihar and A. Bharti, "Importance of software testing in software development life cycle," *Int. J. Res.*, vol. 6, no. 1, pp. 888–900, 2019.
- [2] R. Ranjan, "A review paper on software testing," *Int. Res. J. Modernization Eng. Technol. Sci.*, vol. 6, no. 3, pp. 3074–3083, Mar. 2024.
- [3] N. Chondamrongkul, G. Hristov, and P. Temdee, "Addressing technical challenges in large language model-driven educational software system," *IEEE Access*, vol. 13, pp. 12846–12858, 2025.
- [4] E. Baccour, N. Mhaisen, A. A. Abdellatif, A. Erbad, A. Mohamed, M. Hamdi, and M. Guizani, "Pervasive AI for IoT applications: A survey on resource-efficient distributed artificial intelligence," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2366–2418, 4th Quart., 2022.
- [5] M. Almutawa, Q. Ghabrah, and M. Canini, "Towards LLM-assisted system testing for microservices," in *Proc. IEEE 44th Int. Conf. Distrib. Comput. Syst. Workshops (ICDCSW)*, Jul. 2024, pp. 29–34.
- [6] K. N. Raja Praveen and G. Sudhamsu, "Using AIG in verilog HDL, autonomous testing in a family of Wien bridge cross transducers," in *Proc. 5th Int. Conf. Contemp. Comput. Informat. (IC3I)*, Dec. 2022, pp. 712–715.
- [7] O. Loubiri and S. Maag, "Automated Web testing using machine learning and containerization," in *Proc. 26th Int. Conf. Circuits, Syst., Commun. Comput. (CSCC)*, Jul. 2022, pp. 113–121.
- [8] D. Corradini, Z. Montolli, M. Pasqua, and M. Ceccato, "DeepREST: Automated test case generation for REST APIs exploiting deep reinforcement learning," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 1383–1394.
- [9] Y. Zheng, Z. Feng, Z. Hu, and K. Pei, "MindFI: A fault injection tool for reliability assessment of MindSpore applications," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Oct. 2021, pp. 235–238.
- [10] Z. Liu, Y. Feng, and Z. Chen, "DialTest: Automated testing for recurrent-neural-network-driven dialogue systems," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2021, pp. 115–126.
- [11] A. Gu, N. Jain, W.-D. Li, M. Shetty, Y. Shao, Z. Li, D. Yang, K. Ellis, K. Sen, and A. Solar-Lezama, "Challenges and paths towards AI for software engineering," 2025, *arXiv:2503.22625*.
- [12] S. Rico, "Challenges and opportunities for generative ai in software engineering," *ACM Trans. Softw. Eng. Methodology*, pp. 1338–1344, 2025.
- [13] Y. Zhang, "New approaches to automated software testing based on artificial intelligence," in *Proc. 5th Int. Conf. Artif. Intell. Comput. Eng. (ICAICE)*, Nov. 2024, pp. 806–810.
- [14] S. Ndlovu and E. Mnkandla, "A preliminary framework for optimising test case selection using natural language processing and test case prioritisation using deep learning in continuous integration," in *Proc. 3rd Zimbabwe Conf. Inf. Commun. Technol. (ZCICT)*, Nov. 2024, pp. 1–5.
- [15] F. Ricca, A. Marchetto, and A. Stocco, "A multi-year grey literature review on AI-assisted test automation," 2024, *arXiv:2408.06224*.
- [16] A. Abo-Eleneen, A. Palliyali, and C. Catal, "The role of reinforcement learning in software testing," *Inf. Softw. Technol.*, vol. 164, Dec. 2023, Art. no. 107325.
- [17] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.
- [18] V. H. S. Durelli, R. S. Durelli, S. S. Borges, A. T. Endo, M. M. Eler, D. R. C. Dias, and M. P. Guimarães, "Machine learning applied to software testing: A systematic mapping study," *IEEE Trans. Rel.*, vol. 68, no. 3, pp. 1189–1212, Sep. 2019.
- [19] J. Yoon, R. Feldt, and S. Yoo, "Intent-driven mobile GUI testing with autonomous large language model agents," in *Proc. IEEE Conf. Softw. Test., Verification Validation (ICST)*, May 2024, pp. 129–139.
- [20] C. Peng, Z. Zhang, Z. Lv, and P. Yang, "MUBot: Learning to test large-scale commercial Android apps like a human," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Oct. 2022, pp. 543–552.
- [21] M. Mobilio, D. Clerissi, G. Denaro, and L. Mariani, "GUI testing to the power of parallel Q-learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, May 2023, pp. 55–59.
- [22] Z. Zhang, Y. Liu, S. Yu, X. Li, Y. Yun, C. Fang, and Z. Chen, "UniRLTest: Universal platform-independent testing with reinforcement learning via image understanding," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2022, pp. 805–808.
- [23] M. H. Moghadam, M. Saadatmand, M. Borg, M. Bohlin, and B. Lisper, "An autonomous performance testing framework using self-adaptive fuzzy reinforcement learning," *Softw. Quality J.*, vol. 30, no. 1, pp. 127–159, Mar. 2022.
- [24] B. Arasteh and A. Ghaffari, "A cost-effective and machine-learning-based method to identify and cluster redundant mutants in software mutation testing," *J. Supercomput.*, vol. 80, no. 12, pp. 16711–16743, Aug. 2024.
- [25] O. Rodriguez-Valdes, "Towards a testing tool that learns to test," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, May 2021, pp. 278–280.
- [26] S. Gong, D. Peng, D. Altunbükten, P. Fonseca, and P. Maniatis, "Snowcat: Efficient kernel concurrency testing using a learned coverage predictor," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 35–51.
- [27] P. Pham, V. Nguyen, and T. Nguyen, "A review of AI-augmented end-to-end test automation tools," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2022, pp. 1–4.
- [28] M. R. O. Singh and B. Thankachan, "A detailed survey on machine intelligence based frameworks for software defect prediction," in *Proc. Int. Conf. Comput., Commun., Intell. Syst. (ICCCIS)*, Feb. 2021, pp. 360–365.
- [29] A. Zheng, M. Rana, and A. Stolcke, "Lightweight safety guardrails using fine-tuned BERT embeddings," 2024, *arXiv:2411.14398*.
- [30] Y. Dong, R. Mu, Y. Zhang, S. Sun, T. Zhang, C. Wu, G. Jin, Y. Qi, J. Hu, J. Meng, S. Bensalem, and X. Huang, "Safeguarding large language models: A survey," 2024, *arXiv:2406.02622*.
- [31] M. Monperrus, "Automatic software repair," *ACM Comput. Surveys*, vol. 51, no. 1, pp. 1–24, Jan. 2019.
- [32] Y. Kim and R. Kouatly, "Self-healing automation testing with selenium and ChatGPT API integration to improve the efficiency of the maintenance," in *Proc. 2nd Int. Conf. Found. Large Lang. Models (FLLM)*, Nov. 2024, pp. 339–344.
- [33] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, "A survey of learning-based automated program repair," *ACM Trans. Softw. Eng. Methodology*, vol. 33, no. 2, pp. 1–69, Feb. 2024.
- [34] S. K. Pandey, R. B. Mishra, and A. K. Tripathi, "Machine learning based methods for software fault prediction: A survey," *Expert Syst. Appl.*, vol. 172, Jun. 2021, Art. no. 114595.
- [35] P. Rempel and P. Mäder, "Preventing defects: The impact of requirements traceability completeness on software quality," *IEEE Trans. Softw. Eng.*, vol. 43, no. 8, pp. 777–797, Aug. 2017.
- [36] F. Ferreira, "A test strategy for configurable software systems using machine learning," in *Proc. CIBSE—Ibero Amer. Conf. Softw. Eng. Minas Gerais, Brazil: Federal University of Minas Gerais*, 2020.
- [37] N. Humbatova, G. Jahangirova, and P. Tonella, "DeepCrime: From real faults to mutation testing tool for deep learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, May 2023, pp. 68–72.
- [38] C. Wan, S. Liu, S. Xie, Y. Liu, H. Hoffmann, M. Maire, and S. Lu, "Automated testing of software that uses machine learning APIs," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 212–224.
- [39] H. Yin, H. Mohammed, and S. Boyapati, "Leveraging pre-trained large language models (LLMs) for on-premises comprehensive automated test case generation: An empirical study," in *Proc. 9th Int. Conf. Intell. Informat. Biomed. Sci. (ICIBMS)*, Nov. 2024, pp. 597–607.
- [40] A. Gamal, R. Emad, T. Mohamed, O. Mohamed, A. Hamdy, and S. Ali, "Owl eye: An AI-driven visual testing tool," in *Proc. 5th Novel Intell. Lead. Emerg. Sci. Conf. (NILES)*, Oct. 2023, pp. 312–315.
- [41] M. Ojdanic, A. Khanfir, A. Garg, R. Degiovanni, M. Papadakis, and Y. L. Traon, "On comparing mutation testing tools through learning-based mutant selection," in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Test (AST)*, May 2023, pp. 35–46.
- [42] Q. Han, Z. Shi, and Z. Zhao, "Research on trustworthy software testing techniques based on large models," in *Proc. 10th Int. Symp. Syst. Secur., Saf., Rel. (ISSSR)*, Mar. 2024, pp. 524–525.

- [43] T. Dias, E. Maia, and I. Praça, "FuzzTheREST: An intelligent automated black-box RESTful API fuzzer," in *Proc. Int. Symp. Distrib. Comput. Artif. Intell.*, 2024, pp. 160–169.
- [44] E. Collins, A. Neto, A. Vincenzi, and J. Maldonado, "Deep reinforcement learning based Android application GUI testing," in *Proc. Brazilian Symp. Softw. Eng.*, Sep. 2021, pp. 186–194.
- [45] H. Zhu and I. Bayley, "Exploratory datamorphic testing of classification applications," in *Proc. IEEE/ACM 1st Int. Conf. Autom. Softw. Test*, Oct. 2020, pp. 51–60.
- [46] C. Paduraru, M. Paduraru, and A. Stefanescu, "Optimizing decision making in concolic execution using reinforcement learning," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation Workshops (ICSTW)*, Oct. 2020, pp. 52–61.
- [47] A. Athimoolam and K. Balasubramanian, "Fault recognition of multilevel inverter using artificial neural network approach," *Intell. Autom. Soft Comput.*, vol. 36, no. 2, pp. 1331–1347, 2023.
- [48] M. Ojdanic, A. Garg, A. Khanfir, R. Degiovanni, M. Papadakis, and Y. Le Traon, "Syntactic versus semantic similarity of artificial and real faults in mutation testing studies," *IEEE Trans. Softw. Eng.*, vol. 49, no. 7, pp. 3922–3938, Jul. 2023.
- [49] C. Paduraru, M. Paduraru, and A. Stefanescu, "RiverGame—A game testing tool using artificial intelligence," in *Proc. IEEE Conf. Softw. Test., Verification Validation (ICST)*, Apr. 2022, pp. 422–432.
- [50] W. Zheng, L. Lin, X. Chen, G. Liu, H. Huang, J. Shen, Q. Xu, and Y. Gu, "ISTA: Automatic test case generation and optimization for intelligent systems based on coverage analysis," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Mar. 2023, pp. 758–762.
- [51] L. R. Guerino, P. H. Kuroishi, A. C. R. Paiva, and A. M. R. Vincenzi, "Static and dynamic comparison of mutation testing tools for Python," in *Proc. 23rd Brazilian Symp. Softw. Quality*, Nov. 2024, pp. 199–209.
- [52] A. Guo, Y. Feng, and Z. Chen, "LiRTTest: Augmenting LiDAR point clouds for automated testing of autonomous driving systems," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2022, pp. 480–492.
- [53] A. Thool and C. Brown, "Harnessing the power of LLMs: LLM summarization for human-centric DAST reports," in *Proc. IEEE Symp. Vis. Lang. Hum.-Centric Comput. (VL/HCC)*, vol. 1, Sep. 2024, pp. 33–39.
- [54] K. Ivanova, G. Kondratenko, I. Sidenko, and Y. Kondratenko, "Artificial intelligence in automated system for Web-interfaces visual testing," in *Proc. COLINS*, vol. 2604, 2020, pp. 1019–1031.
- [55] A. Ruospo, D. Piumatti, A. Floridia, and E. Sanchez, "A suitability analysis of software based testing strategies for the on-line testing of artificial neural networks applications in embedded devices," in *Proc. IEEE 27th Int. Symp. On-Line Test. Robust Syst. Design (IOLTS)*, Jun. 2021, pp. 1–6.
- [56] K. Y. Goh, S. Baharom, J. Din, and N. M. Sharef, "The development of an Android automated testing tool: CrashDroid," in *Proc. Appl. Informat. Int. Conf. (AiIC)*, May 2022, pp. 165–171.
- [57] V. Guilherme and A. Vincenzi, "An initial investigation of ChatGPT unit test generation capability," in *Proc. 8th Brazilian Symp. Systematic Automated Softw. Test.*, Sep. 2023, pp. 15–24.
- [58] A. Lops, F. Narducci, A. Ragone, and M. Trizio, "AgoneTest: Automated creation and assessment of unit tests leveraging large language models," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 2440–2441.
- [59] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, Z. Tian, Y. Huang, J. Hu, and Q. Wang, "Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1685–1696.
- [60] M. Hekmatnejad, B. Hoxha, and G. Fainekos, "Search-based test-case generation by monitoring responsibility safety rules," 2020, *arXiv:2005.00326*.
- [61] M. A. Job, "Automating and optimizing software testing using artificial intelligence techniques," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 5, pp. 594–602, 2021.
- [62] M. Padmanabhan, "Test case optimization based on specification diagrams and simulation invocation relationship," in *Proc. 4th Int. Conf. Electr. Comput. Commun. Technol. (ICECCT)*, Sep. 2021, pp. 1–6.
- [63] M. Ali, T. Mazhar, Y. Arif, S. Al-Otaibi, Y. Y. Ghadi, T. Shahzad, M. A. Khan, and H. Hamam, "Software defect prediction using an intelligent ensemble-based model," *IEEE Access*, vol. 12, pp. 20376–20395, 2024.
- [64] R. Jabbarvand, F. Mehralian, and S. Malek, "Automated construction of energy test oracles for Android," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2020, pp. 927–938.
- [65] M. Komar, V. Fedorovich, V. Poidych, and A. Taborovskiy, "Intelligent system for visual testing of software products," in *Proc. 1st Int. Workshop Young Scientists Artif. Intell. Sustain. Develop.*, vol. 3716, 2024, pp. 9–18.
- [66] C. Zhang, T. Shi, J. Ai, and W. Tian, "Construction of GUI elements recognition model for AI testing based on deep learning," in *Proc. 8th Int. Conf. Dependable Syst. Appl. (DSA)*, Aug. 2021, pp. 508–515.
- [67] Y. Wu, J. Su, D. D. Moran, and C. D. Near, "Customized security triage from analysis tools: Current state of the art," in *Proc. 9th Int. Conf. Comput. Commun. (ICCC)*, Dec. 2023, pp. 2632–2637.
- [68] R. Santos, S. Rizvi, B. Cesarone, W. Gunn, and E. McConnell, "Reducing software vulnerabilities using machine learning static application security testing," in *Proc. Int. Conf. Softw. Secur. Assurance (ICSSA)*, Nov. 2021, pp. 43–46.
- [69] C. Şerban and F. Bota, "A conceptual framework for software fault prediction using neural networks," in *Proc. Commun. Comput. Inf. Sci.*, 2020, pp. 171–186.
- [70] J. K. Lee, E. Oh, and H. Shin, "Monte Carlo simulator for threshold voltage distribution of 3-D NAND flash memory using machine learning," *IEEE Trans. Electron Devices*, vol. 71, no. 1, pp. 542–546, Jan. 2024.
- [71] X. Gao, Z. Wang, Y. Feng, L. Ma, Z. Chen, and B. Xu, "Multitest: Physical-aware object insertion for testing multi-sensor fusion perception systems," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1710–1722.
- [72] H. Zhu and I. Bayley, "Discovering boundary values of feature-based machine learning classifiers through exploratory datamorphic testing," *J. Syst. Softw.*, vol. 187, May 2022, Art. no. 111231.
- [73] O. Gordieiev, D. Gordieieva, V. Kharchenko, I. Kondius, and I. Brezhnev, "Extended model of software quality assessment scenario: Concept, operations, application," in *Proc. COLINS*, vol. 3171, 2022, pp. 1629–1638.
- [74] S. Lipp, D. Elsner, S. Kacianka, A. Pretschner, M. Böhme, and S. Banescu, "Green fuzzing: A saturation-based stopping criterion using vulnerability prediction," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 127–139.
- [75] M. Savić, M. Mäntylä, and M. Claes, "Win GUI crawler: A tool prototype for desktop GUI image and metadata collection," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation Workshops (ICSTW)*, Apr. 2022, pp. 223–228.
- [76] J. Olszewska, "AI-T: Software testing ontology for AI-based systems," in *Proc. 12th Int. Joint Conf. Knowl. Discovery, Knowl. Eng. Knowl. Manage.*, 2020, pp. 291–298.
- [77] M. M. Morovati, A. Nikanjam, F. Khomh, and Z. M. Jiang, "Bugs in machine learning-based systems: A faultload benchmark," *Empirical Softw. Eng.*, vol. 28, no. 3, p. 62, May 2023.
- [78] D. J. Fremont, E. Kim, Y. V. Pant, S. A. Seshia, A. Acharya, X. Bruso, P. Wells, S. Lemke, Q. Lu, and S. Mehta, "Formal scenario-based testing of autonomous vehicles: From simulation to the real world," 2020, *arXiv:2003.07739*.
- [79] S. Xie, S. Zhu, Q. He, and Z. Cui, "Autonomous driving software testing based on interpretation analysis," in *Proc. 11th Int. Conf. Dependable Syst. Appl. (DSA)*, Nov. 2024, pp. 432–433.
- [80] K. Zhao, J. Song, L. Sha, H. Shen, Z. Chen, T. Zhao, X. Liang, and J. Yin, "GUI testing arena: A unified benchmark for advancing autonomous GUI testing agent," 2024, *arXiv:2412.18426*.
- [81] Ó. Soto-Sánchez, M. Maes-Bermejo, M. Gallego, and F. Gortázar, "A dataset of regressions in Web applications detected by end-to-end tests," *Softw. Quality J.*, vol. 30, no. 2, pp. 425–454, Jun. 2022.
- [82] J. Wu, Y. Wang, R. Wang, J. Chen, and Z. Zheng, "Can neural networks help smart contract testing? An empirical study," in *Proc. 14th Asia-Pacific Symp. Internetwork*, Aug. 2023, pp. 79–89.
- [83] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, "Fill in the blank: Context-aware automated text input generation for mobile GUI testing," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 1355–1367.
- [84] A. Trudova, M. Dolezel, and A. Buchalceva, "Artificial intelligence in software test automation: A systematic literature review," in *Proc. 15th Int. Conf. Eval. Novel Approaches to Softw. Eng.*, 2020, pp. 181–192.
- [85] M. Islam, F. Khan, S. Alam, and M. Hasan, "Artificial intelligence in software testing: A systematic review," in *Proc. IEEE Region 10 Conf. (TENCON)*, vol. 9, Oct. 2023, pp. 524–529.
- [86] D. S. Battina, "Artificial intelligence in software test automation: A systematic literature review," *J. Emerg. Technol. Innov. Res.*, vol. 6, pp. 1329–1332, Jan. 2019.

- [87] V. Garousi, N. Joy, Z. Jafarov, A. B. Keleş, S. Değirmenci, E. Özdemir, and R. Zarringhalami, "AI-powered software testing tools: A systematic review and empirical assessment of their features and limitations," 2024, *arXiv:2409.00411*.
- [88] E. Naresh, B. P. V. Kumar, M. D. Naik, and S. K. Ahuja, "Survey on test generation using machine learning technique," *Int. J. Recent Technol. Eng.*, vol. 7, no. 6, pp. 562–566, 2019.
- [89] L. Highams and J. I. Olszewska, "Markov-chain-based musical creative intelligent agent passing successfully the Turing test," in *Proc. IEEE 27th Int. Conf. Intell. Eng. Syst. (INES)*, Jul. 2023, pp. 000215–000220.
- [90] M. Ma, L. Han, and Y. Qian, "CVDF DYNAMIC—A dynamic fuzzy testing sample generation framework based on BI-LSTM and genetic algorithm," *Sensors*, vol. 22, no. 3, p. 1265, Feb. 2022.
- [91] H. Sartaj, A. Muqet, M. Z. Iqbal, and M. U. Khan, "Automated system-level testing of unmanned aerial systems," *Automated Softw. Eng.*, vol. 31, no. 2, p. 64, Nov. 2024.



NUNO POMBO (Senior Member, IEEE) is currently an Associate Professor with the University of Beira Interior (UBI), Covilhã, Portugal. He is a member of the Secure and Intelligent Networked Software Systems (sins-lab), Instituto de Telecomunicações (IT). His current research interests include software engineering, information systems (with special focus on decision support systems), and software engineering education.

...



AZAADE FARAJI (Student Member, IEEE) is currently pursuing the Ph.D. degree in computer engineering with the University of Beira Interior (UBI), Covilhã, Portugal. She is also with the HULTIG Laboratory, UBI. She has contributed to the preparation of scientific publications in her field. She was a recipient of the Doctoral Fellowship awarded by the Foundation for Science and Technology (FCT), Portugal. Her research interests include AI-driven software testing, intelligent systems, deep learning, and the computational modeling of cultural heritage markers in children's play behavior across diverse cultural contexts.